

Data Warehouse and Visualization

Analisi dell'E-Commerce Olist

Emanuele Colecchia

Matricola: 276527

`clcmn102c06g317o@studenti.unical.it`

Anno Accademico 2025/2026

Contents

1	Descrizione della sorgente dati e analisi dei requisiti	3
1.1	Fonte dei dati	3
1.2	Analisi preliminare dei requisiti	3
2	Schema del Reconciled Database	4
2.1	Scelte progettuali	4
2.2	Schema Entità-Relazione	4
2.3	Schema Relazionale	5
3	Data Quality Assessment e Data Cleaning	6
3.1	Metodologia	6
3.2	Problemi rilevati e interventi	8
3.2.1	Customers (99.441 record)	8
3.2.2	Sellers (3.095 record)	9
3.2.3	Geolocation (1.000.163 record originali)	10
3.2.4	Products (32.951 record)	11
3.2.5	Orders (99.441 record)	12
3.2.6	Order Items (112.650 record)	13
3.2.7	Payments (103.886 record)	15
3.2.8	Reviews (99.224 record dopo cleaning)	16
3.3	Riepilogo degli interventi di data cleaning	18
4	Progettazione Concettuale – Schema DFM	19
4.1	Scelta del fatto	19
4.2	Attribute Tree	20
4.3	Dall'Attribute Tree allo Schema DFM	20
4.4	Considerazioni sulla perdita di informazione	22
5	Progettazione Logica – Star Schema	23
5.1	Traduzione dal DFM allo Star Schema	23
5.2	Supporto alle operazioni OLAP	24
5.3	Tabelle del Data Warehouse	25
5.4	Glossario delle misure	27
6	Implementazione e Popolamento (ETL)	28

6.1	Architettura del processo ETL	28
6.2	Dettaglio dei JOIN per la costruzione di fact_sales e delle dimensioni . .	29
6.3	Statistiche Finali del Data Warehouse	30

1 Descrizione della sorgente dati e analisi dei requisiti

1.1 Fonte dei dati

Il dataset scelto per questo progetto è il *Brazilian E-Commerce Public Dataset* di Olist, disponibile su Kaggle:

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Olist è un marketplace brasiliano che mette in contatto piccoli venditori con le principali piattaforme di e-commerce del Brasile. Il dataset contiene circa 100.000 ordini reali, anonimizzati, effettuati tra il 2016 e il 2018. È distribuito in nove file CSV separati, descritti nella Tabella 1.

Table 1: File CSV della sorgente dati Olist

File	Contenuto
olist_customers_dataset.csv	Anagrafica clienti con localizzazione geografica
olist_sellers_dataset.csv	Anagrafica venditori con localizzazione geografica
olist_orders_dataset.csv	Ordini con stati e date del ciclo di vita
olist_order_items_dataset.csv	Prodotti acquistati per ordine, con prezzi e spedizioni
olist_order_payments_dataset.csv	Metodi e valori di pagamento
olist_order_reviews_dataset.csv	Recensioni e valutazioni dei clienti
olist_products_dataset.csv	Catalogo prodotti con dimensioni e categorie
olist_geolocation_dataset.csv	Coordinate geografiche per prefisso postale
product_category_name_translation.csv	Traduzione categorie da portoghese a inglese

1.2 Analisi preliminare dei requisiti

L'obiettivo è costruire un sistema di supporto alle decisioni per analizzare le performance commerciali di Olist. Le domande di business principali a cui il data warehouse deve rispondere sono:

1. Quali categorie di prodotto generano più fatturato? Come si distribuisce geograficamente?
2. Come sono evolute le vendite nel tempo? Ci sono picchi stagionali rilevanti?
3. Come varia la soddisfazione dei clienti (punteggio recensioni) per categoria di prodotto, stato brasiliano e venditore?
4. Che percentuale di ordini viene consegnata in tempo? Qual è il ritardo medio e come cambia per zona geografica?
5. Quali metodi di pagamento preferiscono i clienti?

Per rispondere servono le seguenti **dimensioni**: Tempo (giornaliera, fino all'anno), Prodotto (per categoria e macro-categoria), Cliente (geografica: città–stato–regione), Venditore (geografica: città–stato), Metodo di Pagamento. Le **misure** principali sono: fatturato, costo di spedizione, numero di item venduti, punteggio recensioni, ritardo di consegna.

Nota implementativa: le gerarchie `macro_category` (Prodotto) e `region` (Cliente) sono previste nel modello concettuale DFM ma non implementate nella versione corrente dello star schema, poiché la sorgente non include una tabella di mapping categoria→macro-categoria né una corrispondenza stato→regione geografica. Tutte le altre dimensioni e misure sono implementate integralmente.

2 Schema del Reconciled Database

2.1 Scelte progettuali

A partire dai CSV, ho progettato un reconciled database relazionale che consolida i dati sorgente in uno schema normalizzato e coerente. Il reconciled database rappresenta i dati operativi ottenuti dopo l'integrazione e la pulizia delle sorgenti: i dati risultano integrati, consistenti, corretti e dettagliati, pronti per essere caricati nel data warehouse tramite il processo ETL.

Rispetto alla struttura originale, ho introdotto tre modifiche principali:

- calcolo degli attributi derivati `delivery_lead_days` e `delivery_delay_days` nella tabella `Orders`;
- aggiunta di `product_category_name_english` in `Products` tramite join con la tabella di traduzione;
- aggregazione di `Geolocation` per prefisso postale (un record per ZIP), calcolando il centroide delle coordinate e la città/stato più frequenti.

2.2 Schema Entità-Relazione

Lo schema E/R è illustrato nella Figura 1.

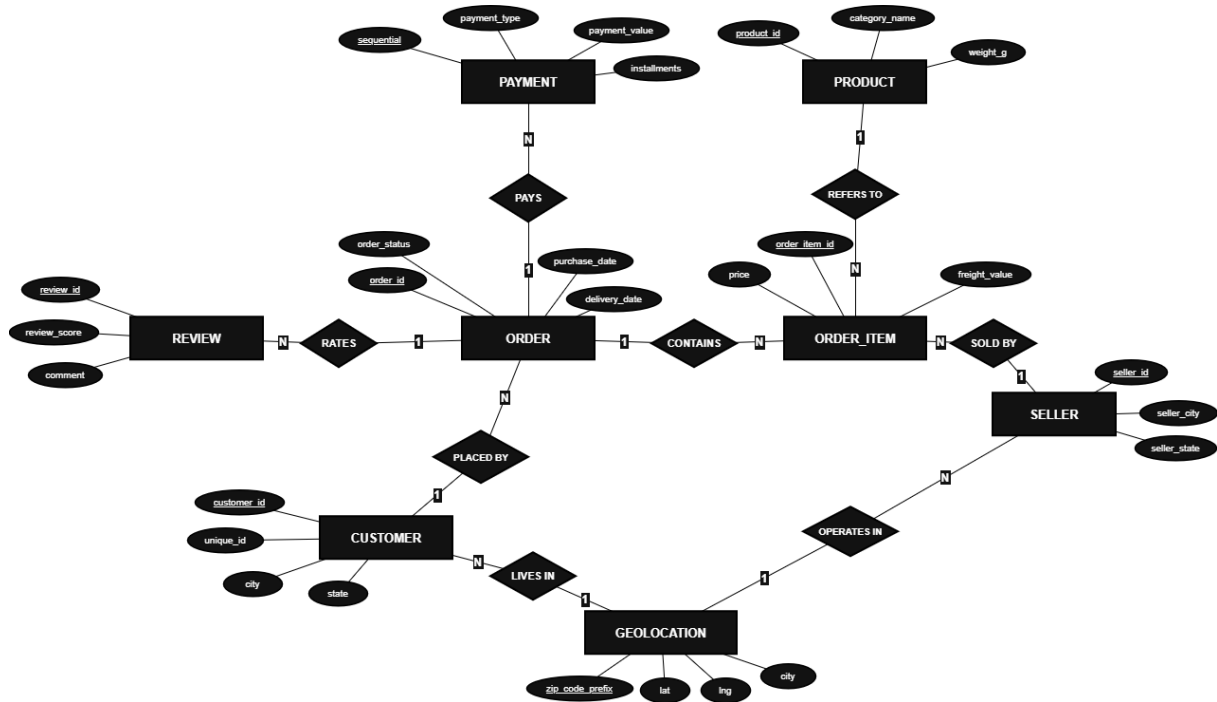


Figure 1: Schema E/R del Reconciled Database

2.3 Schema Relazionale

Di seguito lo schema relazionale (chiavi primarie sottolineate; asterisco = chiave esterna):

- **Customers**(customer_id, customer_unique_id, zip_code_prefix*, city, state)
customer_unique_id identifica il cliente reale: uno stesso cliente può avere più *customer_id* su ordini distinti. *zip_code_prefix* è FK parziale/opzionale verso *Geolocation* (278 CAP clienti assenti dalla tabella di geolocalizzazione; join eseguito come *LEFT JOIN*).
- **Sellers**(seller_id, zip_code_prefix*, city, state)
zip_code_prefix è FK parziale/opzionale verso *Geolocation* (7 CAP venditori assenti; join eseguito come *LEFT JOIN*).
- **Products**(product_id, category_name, category_name_english, name_length, description_length, photos_qty, weight_g, length_cm, height_cm, width_cm)
category_name_english è attributo derivato dal join con la tabella di traduzione.
- **Orders**(order_id, customer_id*, status, purchase_timestamp, approved_at, delivered_carrier_date, delivered_customer_date, estimated_delivery_date, delivery_lead_days, delivery_delay_days)
delivery_lead_days e *delivery_delay_days* sono calcolati in fase di cleaning. Sono *NULL* per ordini non ancora consegnati; questa condizione è tracciata da appositi flag binari.
- **Order_Items**(order_id*, order_item_id, product_id*, seller_id*, shipping_limit_date, price, freight_value)
 Chiave primaria composta: un ordine può contenere più prodotti.

- **Payments**(order_id*, payment_sequential, payment_type, installments, value)
Un ordine può avere più metodi di pagamento, da cui la chiave composta con `payment_sequential`.
- **Reviews**(review_id, order_id*, score, comment_title, comment_message, creation_date, answer_timestamp)
Il punteggio è un intero da 1 a 5. I campi testo sono facoltativi.
- **Geolocation**(zip_code_prefix, lat, lng, city, state)
Mappa ogni prefisso postale a un centroide geografico. Risultato dell'aggregazione per ZIP effettuata in cleaning.

3 Data Quality Assessment e Data Cleaning

3.1 Metodologia

Le operazioni di pulizia sono state implementate in un notebook Python (`olist_dw_cleaning_pipeline`) strutturato come una pipeline modulare e auditabile. Ogni fase di trasformazione registra un entry nell'audit log globale, che traccia per ogni step: tabella, operazione, righe prima/dopo, numero di modifiche e note esplicative.

Il processo ha seguito le fasi dello standard di data quality assessment adottato a lezione: **analisi dei dati grezzi** (profiling), **identificazione dei problemi** con scorecard programmatica, **classificazione** secondo la tassonomia del corso e **intervento correttivo**, seguito dalla verifica dei risultati.

Classificazione dei problemi di data quality

Secondo la tassonomia adottata nel corso, i problemi di qualità dei dati si distinguono in:

- **Missing values**: valori assenti che devono essere presenti per garantire la completezza del dato. Da distinguere dai *null semantici*, che sono assenze legittime (es. data di consegna per ordini non ancora consegnati).
- **Wrong values**: valori presenti ma scorretti rispetto al dominio atteso (es. `customer_zip_code_prefix` troncato a 4 cifre per perdita dello zero iniziale, coordinate fuori range geografico).
- **Noisy data**: valori plausibili ma anomali rispetto alla distribuzione statistica, non rilevabili da regole di dominio semplici. Richiedono analisi della distribuzione (es. varianza/deviazione standard, media $\pm 3\sigma$).
- **Violations of attribute dependencies**: valori che violano dipendenze funzionali tra attributi dello stesso record (es. `approved_at < purchase_timestamp`).
- **Record duplication**: più record che rappresentano la stessa entità reale, con o senza differenze di valore.
- **Varying value representations**: lo stesso concetto rappresentato con formati diversi (es. “são paulo” vs. “São Paulo”).

Indici di data quality

Per ogni dataset sono stati misurati i seguenti cinque indici di qualità:

- **Validity:** percentuale di valori che rispettano le regole di dominio dichiarate (tipo, range, whitelist). Calcolata per attributo.
- **Completeness:** percentuale di valori non nulli sugli attributi obbligatori. Non include attributi facoltativi per non falsare l'indice.
- **Uniqueness:** percentuale di record non duplicati sulla chiave primaria.
- **Consistency:** percentuale di record che rispettano le dipendenze funzionali inter-attributo dichiarate.
- **Precision:** percentuale di dati corretti, ovvero non anomali statisticamente. Per colonne numeriche è calcolata come percentuale di valori entro media $\pm 3\sigma$; per tabelle senza colonne numeriche rilevanti, Precision è assunta al 100%.

Gli indici sono calcolati automaticamente da una **scorecard programmatica** (classe `DQAResult`), basata su regole dichiarative per ciascun attributo (lambda Python per Validity, lista attributi obbligatori per Completeness, chiave per Uniqueness, lista di dipendenze per Consistency). La scorecard fornisce una vista oggettiva e riproducibile della qualità, ma non è sufficiente da sola: i *noisy data* non emergono dagli indici di Validity se le regole sono permissive (es. `price > 0`), e richiedono un'ispezione aggiuntiva delle distribuzioni statistiche.

IA – Data Quality Assessment (L1) – 10_script_llm

A supporto dell'interpretazione dei risultati della scorecard è stato utilizzato un Large Language Model (llama3.2:3b via Ollama, locale) come strumento di analisi narrativa. Per ogni tabella, dopo aver calcolato il profilo statistico con pandas (null per colonna, duplicati, statistiche numeriche min/max/mean), il prompt inviato al modello era: *“Analizza la qualità dei dati di questa tabella Olist:”* seguito dal profilo calcolato. Il LLM ha prodotto per ogni tabella una descrizione in linguaggio naturale dei problemi rilevati, con una spiegazione del perché ciascun problema impatta il Data Warehouse e un suggerimento operativo. Questo ha supportato le decisioni di intervento descritte nei paragrafi seguenti, nei casi in cui la scorecard da sola non era sufficiente a spiegare la natura del problema.

La Tabella 2 riassume le dimensioni delle tabelle sorgente prima di qualsiasi intervento.

Table 2: Dimensioni delle tabelle sorgente (dati grezzi)

Tabella	Righe	Missing totali	Duplicati esatti
Customers	99.441	0	0
Sellers	3.095	0	0
Products	32.951	4.456	0
Orders	99.441	4.908	0
Order Items	112.650	0	0
Payments	103.886	0	0
Reviews	99.224	145.903	0
Geolocation	1.000.163	0	261.831

3.2 Problemi rilevati e interventi

3.2.1 Customers (99.441 record)

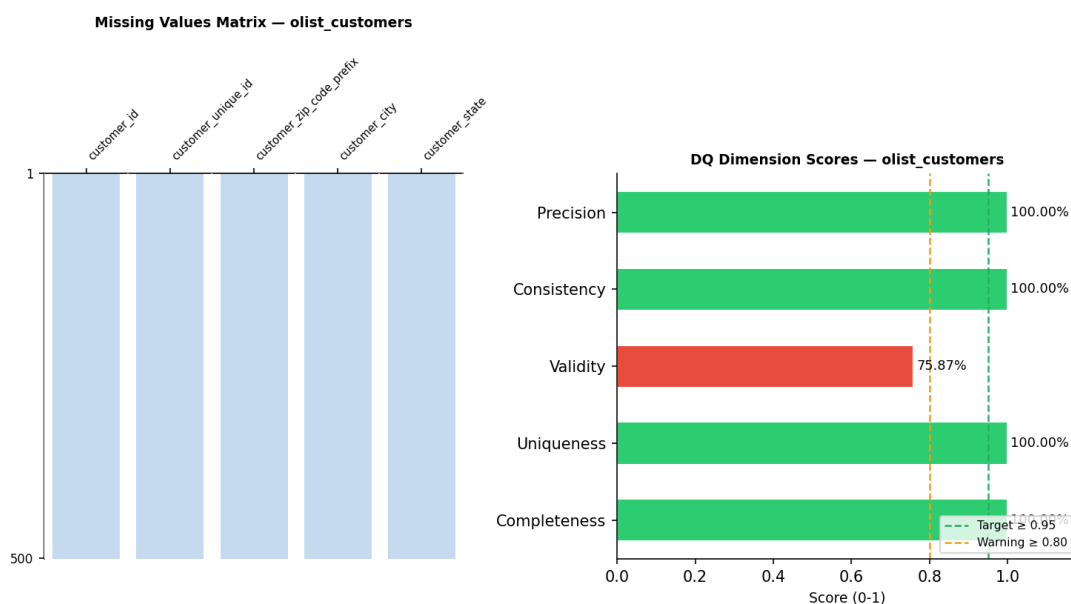


Figure 2: Data quality assessment – olist_customers_dataset

Completeness, Uniqueness e Consistency sono al 100%. Lo scorecard rileva un problema di **Validity** (75,87%), interamente attribuibile alla colonna `customer_zip_code_prefix`: la regola di dominio richiede un CAP numerico a 5 cifre, ma 23.995 record su 99.441 (24,1%) presentano un CAP a 4 cifre, perché il CSV originale memorizza il campo come numero e perde lo zero iniziale dei CAP che iniziano per 0 (*wrong values*). La colonna `customer_state` risulta invece già conforme alla whitelist `BR_STATES` (0 record invalidi): nel CSV originale i codici di stato sono già tutti in maiuscolo.

Un'ispezione manuale rivela inoltre un secondo problema, non catturato dallo scorecard perché `customer_city` non è soggetto a regole di dominio: la presenza di **varying value representations** sui nomi di città (es. “são paulo” vs. “São Paulo”). Si tratta di un errore

a livello di istanza classificabile come problema di **Consistency**, gestito a parte rispetto allo scorecard.

Intervento – automatic correction: `str.strip()` su tutti i campi testo per rimuovere spazi superflui; `str.title()` su `customer_city` (Consistency); `str.upper()` su `customer_state` applicato in via cautelativa per garantire l'invarianza rispetto al maiuscolo/minuscolo (0 modifiche effettive, poiché i valori erano già tutti conformi a `BR_STATES`).

Risultato: 99.441 record, 0 missing, 0 duplicati; Completeness, Uniqueness e Consistency al 100%. Il problema di Validity sul `customer_zip_code_prefix` (CAP a 4 cifre) non viene corretto dalla pipeline: resta documentato come limite noto della qualità del dato, senza impatto sulle analisi a livello città/stato.

3.2.2 Sellers (3.095 record)

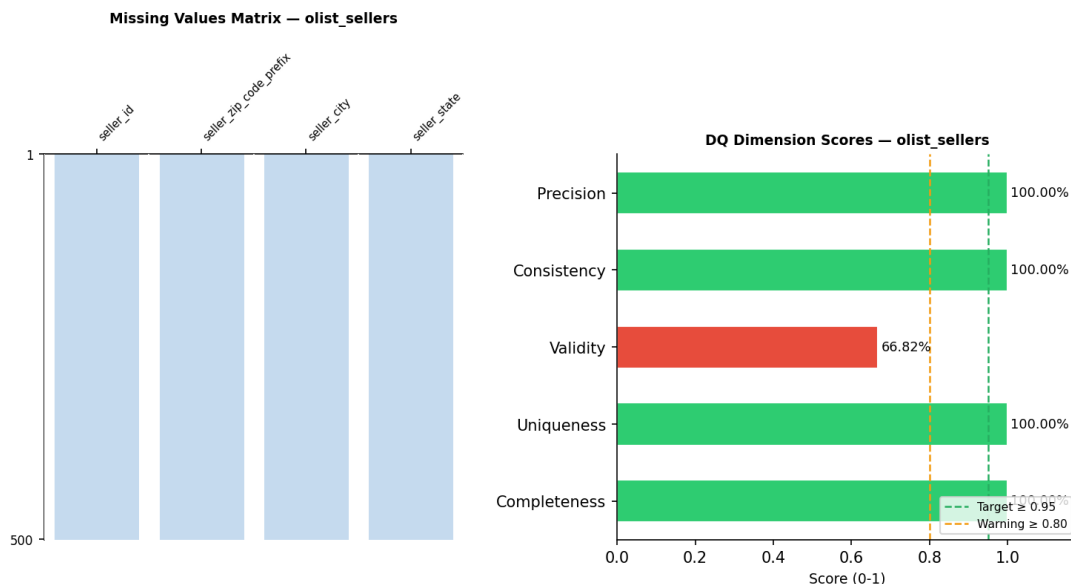


Figure 3: Data quality assessment – olist_sellers_dataset

Situazione analoga a customers: lo scorecard rileva un problema di **Validity** (66,82%), causato dalla colonna `seller_zip_code_prefix`: 1.027 record su 3.095 (33,2%) presentano un CAP a 4 cifre per lo stesso motivo (perdita dello zero iniziale nel CSV originale). La colonna `seller_state` è già conforme alla whitelist `BR_STATES` (0 record invalidi): anche per i sellers i codici di stato sono già tutti in maiuscolo nel CSV originale. L'ispezione manuale rileva inoltre il problema di Consistency sulle *varying value representations* di `seller_city`.

Intervento – automatic correction: `str.strip()` e `str.title()` su `seller_city` (Consistency); `str.upper()` su `seller_state` applicato in via cautelativa (0 modifiche effettive, valori già conformi a `BR_STATES`).

Risultato: 3.095 record, 0 missing, 0 duplicati; Completeness, Uniqueness e Consistency al 100%. Come per customers, il problema di Validity sul `seller_zip_code_prefix` (CAP a 4 cifre) resta un limite noto, documentato ma non corretto dalla pipeline.

3.2.3 Geolocation (1.000.163 record originali)

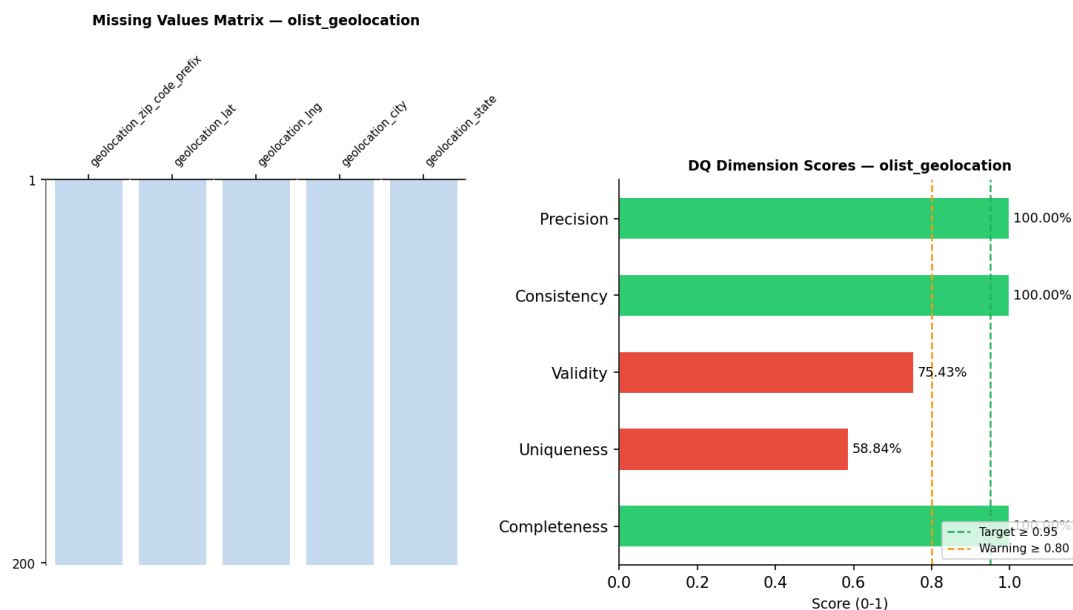


Figure 4: Data quality assessment – olist_geolocation_dataset

Lo scorecard evidenzia due problemi principali. Il primo è di **Uniqueness** (58,84%): il dataset registra più coordinate per lo stesso prefisso postale, con 261.831 righe duplicate (26,2%). Si tratta di *record duplication*: più record rappresentano la stessa entità reale con valori diversi. Il secondo problema è di **Validity** (75,43%): alcune righe presentano coordinate *lat*/*lng* fuori dal range geografico del Brasile ($lat \in [-33.75; 5.27]$, $lng \in [-73.99; -34.79]$), classificabili come *wrong values*.

Interventi:

1. Normalizzazione testuale di **city** e **state** – automatic correction (Consistency).
2. Filtraggio dei record con coordinate fuori range – automatic correction (Validity).
3. Rimozione dei duplicati esatti con `drop_duplicates()` (Uniqueness).
4. Aggregazione per **zip_code_prefix**: media di *lat*/*lng* (centroide), moda di *city*/*state*, mantenendo in **n_records** il conteggio dei punti originali aggregati per ciascun prefisso (utile come indicatore di affidabilità del centroide).

Risultato: 19.015 record, 0 missing, 0 duplicati.

3.2.4 Products (32.951 record)

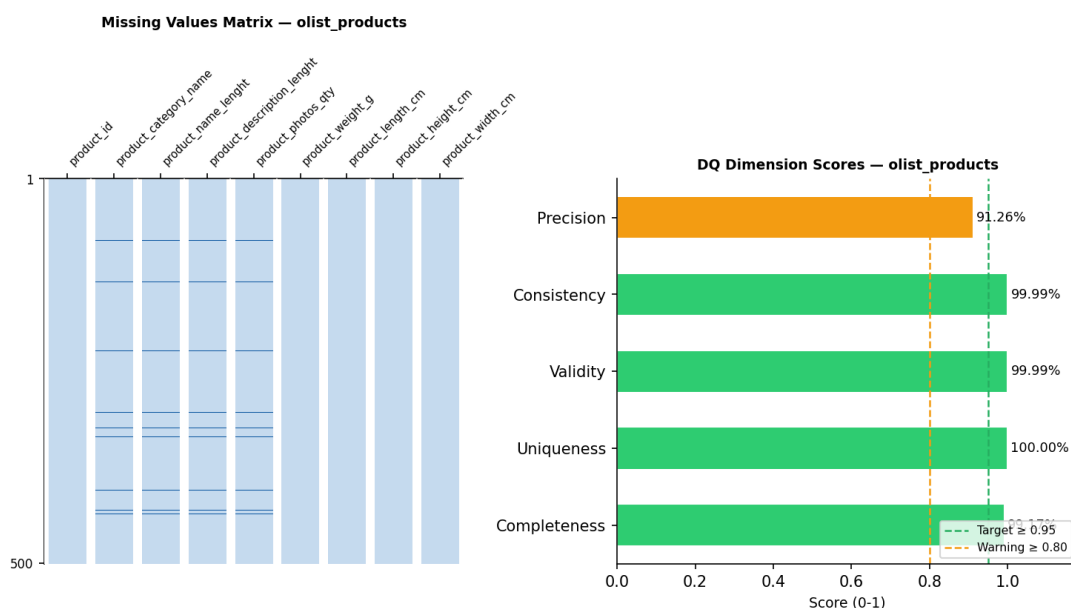


Figure 5: Data quality assessment – olist_products_dataset

Lo scorecard mostra **Completeness** a 99,17%, dovuta a 610 valori mancanti in `product_category_name` (1,85%) e a valori nulli nelle colonne fisiche (peso, dimensioni). Validity, Uniqueness e Consistency sono prossime al 100%.

L'analisi della distribuzione delle misure fisiche rivela inoltre un problema di *noisy data* non rilevato dallo scorecard: la regola di Validity impone solo > 0 , e quindi i valori plausibili ma anomali (peso = 40.000 g, dimensioni estreme) passano il check. Questi outlier costituiscono comunque un problema da trattare, classificabile come *noisy data* / *wrong values* secondo la classificazione del corso, anche se rimangono "invisibili" alla scorecard.

Da notare che i nomi di colonna `product_name_lenght` e `product_description_lenght` (typo nella sorgente: manca la "h") sono stati mantenuti tali quali per compatibilità con lo schema originale.

IA – Missing Values & Outlier Classification (L2) – 10_script_llm

Per la classificazione dei valori mancanti è stato usato il LLM come supporto. Il prompt inviato al modello era: *"Per ogni colonna: classifica MCAR/MAR/MNAR con motivazione, impatto sul DW, strategia raccomandata."*, preceduto dai dati di missingness calcolati con pandas (numero di null e percentuale per colonna). Il modello ha spiegato che i null in `product_category_name` sono classificabili come **MCAR** (Missing Completely At Random): l'assenza non è correlata ad altri attributi ma riflette prodotti non catalogati dal venditore al momento dell'inserimento, senza un pattern sistematico. Ha quindi indicato il *fill in* con costante globale ("**unknown**") come strategia raccomandata, in quanto non introduce bias e preserva la riga nel dataset. Per le misure fisiche, ha suggerito l'imputazione con la mediana di colonna, più robusta della media in presenza di outlier.

Interventi:

1. **Missing values** su `product_category_name` (Completeness): *fill in* automaticamente con la costante globale "**unknown**", poiché non è possibile inferire la categoria corretta senza informazioni aggiuntive (tecnica vista a lezione: *fill in with a global constant*).
2. **Missing values** nelle colonne numeriche fisiche (Completeness): *fill in* automaticamente con la **mediana dell'attributo** calcolata sui record validi. Le slide del corso indicano la media come opzione standard; ho scelto la mediana perché più robusta in presenza di outlier. Per ogni colonna imputata è stato aggiunto un flag binario `_imputed`.
3. **Noisy data** nelle colonne fisiche: rilevamento tramite metodo **IQR** (*Interquartile Range*: soglia inferiore $Q_1 - 1.5 \cdot IQR$, soglia superiore $Q_3 + 1.5 \cdot IQR$); flag binario `_outlier` aggiunto per ogni colonna, valori originali preservati nel reconciled database.
4. Join con la tabella di traduzione per aggiungere `product_category_name_english`.

Risultato: 32.951 record, 0 missing, 0 duplicati. La tabella include 22 colonne (9 originali + colonna EN + 12 flag: 8 flag `_imputed` e 4 flag `_outlier`).

3.2.5 Orders (99.441 record)

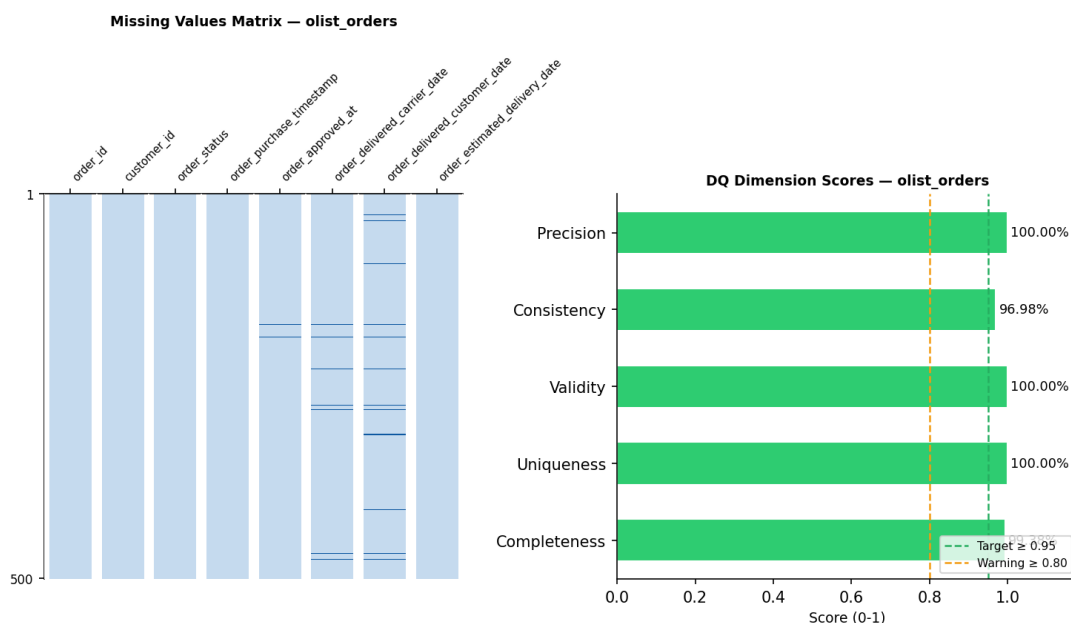


Figure 6: Data quality assessment — olist_orders_dataset

Lo scorecard mostra **Completeness** a 99,38% e **Consistency** a 96,98%, mentre Validity e Uniqueness sono al 100%. La Precision è al 100% per questa tabella, in quanto non sono presenti colonne numeriche con distribuzioni anomale rilevanti; i problemi di noisy data per Orders riguardano le colonne temporali, già gestite tramite flag per i null semantici.

I valori nulli nelle colonne temporali (160 in `order_approved_at`, 1.783 in `order_delivered_carrier_date`, 2.965 in `order_delivered_customer_date`) sono **semanticamente corretti**: corrispondono a ordini non approvati o non ancora consegnati. Non si tratta di veri missing values e non devono essere imputati. Il calo di Consistency (96,98%) è dovuto a 3.005 record che violano almeno una delle dipendenze tra date (es. `approved_at < purchase_timestamp`) – problema classificabile come *violated attribute dependencies* a livello di record.

Interventi:

1. `str.lower()` su `order_status` – automatic correction (Consistency a livello di rappresentazione).
2. `pd.to_datetime()` su tutte le colonne temporali – automatic correction (Validity: i timestamp erano memorizzati come stringhe anziché come date).
3. Flag binari `_approved_missing`, `_delivered_carrier_missing`, `_delivered_customer_missing` per tracciare i null semantici senza imputarli.
4. Le violazioni di dipendenza tra date vengono mantenute nel reconciled database con i timestamp originali (non imputate), ma tracciate per successive analisi.
5. Feature engineering:
 - `delivery_delay_days` = consegna effettiva – consegna stimata (negativo = anticipo);
 - `delivery_lead_days` = consegna effettiva – data acquisto;
 - `purchase_year`, `purchase_month` estratti dal timestamp di acquisto.

Risultato: 99.441 record, 0 duplicati. Rimangono NaN nelle colonne derivate per gli ordini non consegnati (previsto). 15 colonne totali.

3.2.6 Order Items (112.650 record)

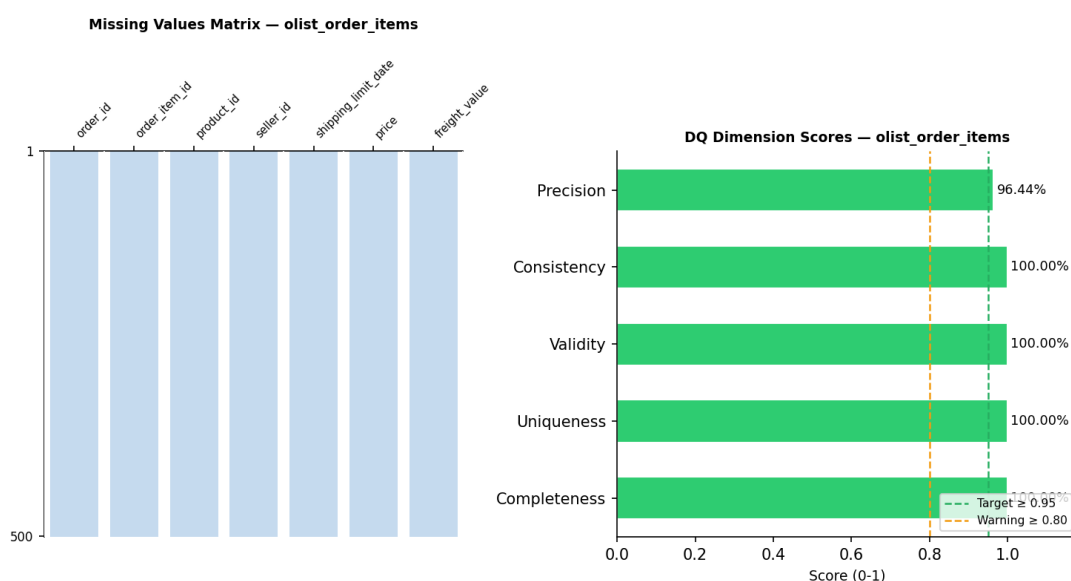


Figure 7: Data quality assessment – `olist_order_items_dataset`

Lo scorecard mostra Completeness, Uniqueness, Validity e Consistency tutte al 100%. La Precision è calcolata sulla colonna **price**: misura la percentuale di valori entro media $\pm 3\sigma$, identificando i potenziali noisy data.

L'analisi della distribuzione di **price** rivela un problema di *noisy data* non rilevato dallo scorecard: la regola di Validity impone solo **price** > 0, e i valori sono tutti positivi, ma la distribuzione si estende fino a 6.735 BRL con una coda destra molto allungata. Stessa situazione per **freight_value**. Si tratta di un classico esempio di problema visibile dall'analisi della distribuzione ma non da una regola puntuale, classificabile come *noisy data* / *wrong values* secondo la tassonomia del corso.

Interventi:

1. `pd.to_datetime()` su `shipping_limit_date` – automatic correction (Validity).
2. **Noisy data** su **price**: rilevamento tramite metodo **IQR** ($Q_1 - 1.5 \cdot IQR$ / $Q_3 + 1.5 \cdot IQR$). Flag binario `_price_outlier` aggiunto. Valori originali preservati.
3. **Noisy data** su **freight_value**: stessa metodologia di **price**. Flag `_freight_value_outlier`. Valori originali preservati.

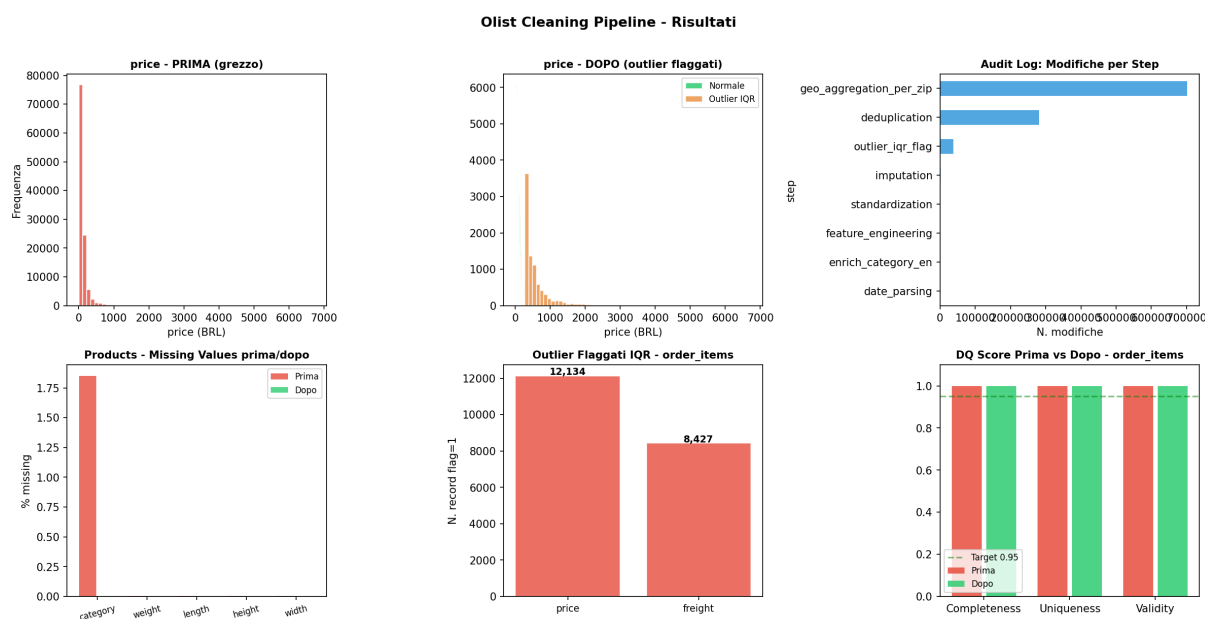


Figure 8: Risultati complessivi della cleaning pipeline: distribuzione **price** prima/dopo, audit log per step, missing values Products, outlier flaggati outlier (metodo IQR) per **order_items** e DQ score prima/dopo.

Risultato: 112.650 record, 0 missing, 0 duplicati. 9 colonne (7 originali + 2 flag `_outlier`).

Nota: `total_item_value = price + freight_value` non è stata aggiunta nel reconciled database perché è una misura derivata semplice; viene calcolata direttamente in Tableau durante la visualizzazione.

3.2.7 Payments (103.886 record)



Figure 9: Data quality assessment – olist_order_payments_dataset

Lo scorecard mostra tutti gli indici al 100%. Il campo `payment_type` ha cinque valori: `credit_card` (76.795, 73,9%), `boleto` (19.784, 19,0%), `voucher` (5.775, 5,6%), `debit_card` (1.529, 1,5%) e `not_defined` (3 record, 0,003%). I tre record `not_defined` sono ammessi nella whitelist di Validity perché il numero è trascurabile e potrebbero essere transazioni legittime non classificate.

Anche qui, l'analisi della distribuzione di `payment_value` evidenzia un problema di *noisy data* che la regola ≥ 0 non cattura: sono presenti valori molto elevati nella coda destra. Lo trattiamo come per i prezzi degli order items.

Interventi:

1. `str.lower()` su `payment_type` – automatic correction (Consistency a livello di rappresentazione).
2. **Noisy data** su `payment_value`: rilevamento tramite metodo **IQR** ($Q_1 - 1.5 \cdot IQR$ / $Q_3 + 1.5 \cdot IQR$) + `flag_payment_value_outlier`. Valori originali preservati.

Risultato: 103.886 record, 0 missing, 0 duplicati. 6 colonne (5 originali + 1 `flag_payment_value_outlier`).

3.2.8 Reviews (99.224 record dopo cleaning)

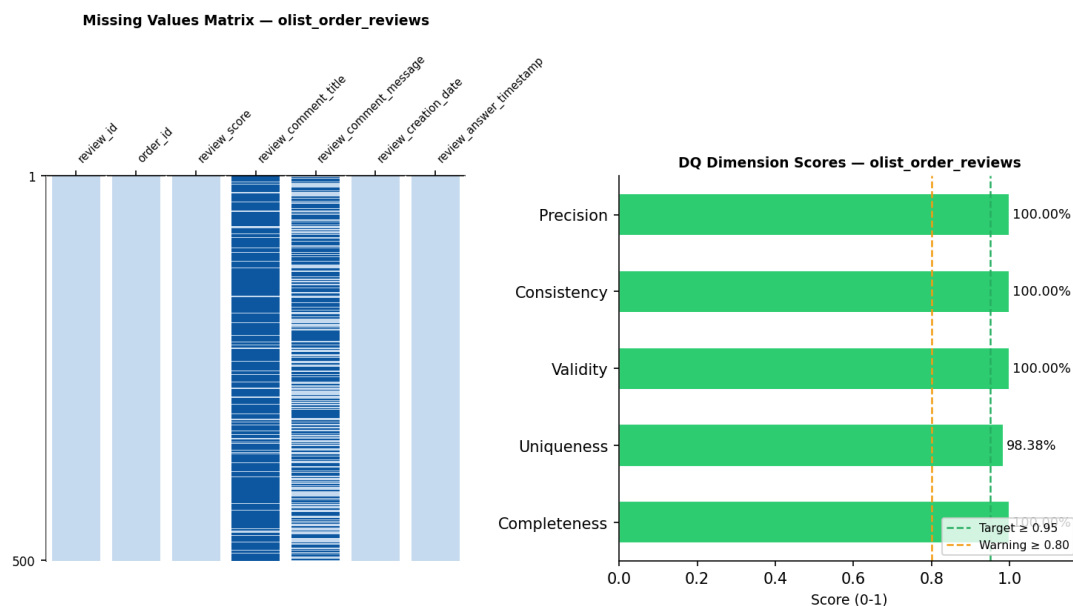


Figure 10: Data quality assessment – olist_order_reviews_dataset

Lo scorecard, calcolato assumendo `review_id` come chiave primaria, mostra **Uniqueness** al 98,38% (1.603 record con `review_id` ripetuto, di cui 814 eccedenti la prima occorrenza), mentre Validity e Completeness sono al 100%.

Un'analisi più approfondita mostra però che questi record non sono duplicati: i 1.603 record con `review_id` ripetuto corrispondono a 789 valori di `review_id` ciascuno associato a 2 o più `order_id` distinti (lo stesso punteggio/commento del cliente viene replicato su più ordini, ad es. ordini multipli o sostituzioni). Sulla chiave composta (`review_id`, `order_id`) non esiste invece alcun duplicato: tutti i 99.224 record sono univoci su questa coppia. La chiave naturale corretta per questa tabella è quindi (`review_id`, `order_id`) e non il solo `review_id`; lo scorecard, basato su una chiave a singolo attributo, sovrastima la duplicazione reale.

È importante notare che il check di Completeness è stato applicato solo agli attributi obbligatori (`review_id`, `order_id`, `review_score`, `review_creation_date`, `review_answer_timestamp`) ed esclude i due campi facoltativi `review_comment_title` e `review_comment_message`.

I null in `review_comment_title` (87.656, 88,3%) e `review_comment_message` (58.247, 58,7%) sono infatti semanticamente corretti: il commento scritto è facoltativo. Includerli nel check di Completeness avrebbe falsato l'indice, perché i null in questi campi non sono "dati mancanti" ma "dati assenti per scelta dell'utente".

Interventi:

1. Deduplicazione su chiave composta (`review_id`, `order_id`) con `drop_duplicates(subset=['review_id', 'order_id'])` (Uniqueness): 0 record rimossi, perché la tabella non contiene duplicati su questa chiave. Una deduplicazione sul solo `review_id` avrebbe invece rimosso erroneamente 814 record legittimi appartenenti ad ordini diversi.

2. `pd.to_datetime()` su `review_creation_date` e `review_answer_timestamp` – automatic correction (Validity).
3. I campi `review_comment_title` e `review_comment_message` con valore nullo sono stati sostituiti con stringa vuota (`fillna("")`): l'assenza del commento è semanticamente corretta e la stringa vuota ne preserva il significato in modo compatibile con i sistemi di destinazione.
4. Verifica del vincolo di dominio su `review_score` (Validity): tutti i valori erano compresi tra 1 e 5, nessuna correzione necessaria.

Risultato: 99.224 record (0 duplicati), invariato rispetto al dataset sorgente; 0 duplicati residui sulla chiave composta (`review_id`, `order_id`).

Impatto a valle su `fact_sales`: i 814 record di recensione recuperati appartengono a 690 ordini distinti che in precedenza, a causa della chiave errata, perdevano una o più valutazioni nella media `AVG(review_score)` per ordine (cfr. Sezione 6.2). Dopo la correzione, `olist_dw.db` è stato ricostruito propagando `fact_reviews.csv` aggiornato: 690 righe di `fact_sales` hanno ora un `review_score` diverso, di cui 549 passano da `NULL` a un valore valido (il totale di righe con `review_score` non nullo sale da 111.159 a 111.708 su 112.650).

3.3 Riepilogo degli interventi di data cleaning

Table 3: Riepilogo degli interventi di data cleaning

Dataset	Problema	Indice	Intervento
Customers / Sellers	CAP a 4 cifre, zero iniziale perso (wrong values)	Validity	Limite noto, non corretto (<code>state.upper()</code> : cautelativo, 0 modifiche)
Customers / Sellers	Varying value representations su city	Consistency	Automatic correction (<code>title()</code> , <code>strip()</code>)
Geolocation	Coordinate fuori range Brasile	Validity	Filtraggio record
Geolocation	Record duplication (261.831)	Uniqueness	<code>drop_duplicates()</code> ; aggregazione per ZIP
Products	Missing values (categoria, misure)	Completeness	Fill in: costante globale "unknown"; mediana dell'attributo
Products	Noisy data (misure fisiche)	– (analisi distribuzione)	Rilevamento IQR ($Q_1/Q_3 \pm 1.5 \cdot IQR$); flag <code>_outlier</code> (valori preservati)
Orders	Timestamp come stringhe	Validity	Automatic correction (<code>to_datetime()</code>)
Orders	Violazioni di dipendenze tra date	Consistency	Mantenuti, tracciati con flag
Order Items	Noisy data (price, freight)	– (analisi distribuzione)	Rilevamento IQR ($Q_1/Q_3 \pm 1.5 \cdot IQR$); flag <code>_outlier</code> (valori preservati)
Payments	Noisy data (payment_value)	– (analisi distribuzione)	Rilevamento IQR ($Q_1/Q_3 \pm 1.5 \cdot IQR$); flag <code>_outlier</code> (valori preservati)
Reviews	<code>review_id</code> non univoco (789 valori su ≥ 2 <code>order_id</code>)	Uniqueness	Dedup. su chiave composta (<code>review_id</code> , <code>order_id</code>); 0 rimossi

Table 4: Dimensioni finali nel reconciled database (dopo cleaning)

Tabella	Righe	Colonne	Note
rec_customers	99.441	5	Invariato
rec_sellers	3.095	4	Invariato
rec_products	32.951	22	+1 EN category, +8 flag <code>_imputed</code> , +4 flag <code>_outlier</code>
rec_geolocation	19.015	6	Aggregata per ZIP
rec_orders	99.441	15	+7 colonne derivate e flag
rec_order_items	112.650	9	+2 flag outlier
rec_fact_payments	103.886	6	+1 flag outlier
rec_fact_reviews	99.224	7	Invariato (chiave: review_id+order_id)

Nota – dove sono state calcolate le colonne aggiunte: tutte le colonne derivate, i flag di qualità e le aggregazioni elencati in questa tabella sono generati dal notebook `olist_dw_cleaning_pipeline.ipynb` (cartella `2_scripts`) e tracciati riga per riga in `audit_log.csv` (tabella, operazione, righe prima/dopo, numero di modifiche). In particolare, le +7 colonne aggiunte a `rec_orders` (cfr. Sezione 3.2.5 per il dettaglio di ciascun intervento) sono:

- `_approved_missing`, `_delivered_carrier_missing`, `_delivered_customer_missing` – flag binari sui null semantici;
- `delivery_lead_days`, `delivery_delay_days` – feature engineering sulle date di consegna;
- `purchase_year`, `purchase_month` – estratti dal timestamp di acquisto.

IA – Cleaning Plan & Audit Narrative (L3) – 10_script_llm

Prima di eseguire il cleaning, il LLM è stato usato come supporto per pianificare gli interventi su ogni tabella. Il prompt inviato era: “*Produci un piano di cleaning per la tabella Olist {nome}*”, accompagnato dal profilo dei dati sporchi (percentuali di null, duplicati, statistiche numeriche). Il modello ha restituito un piano strutturato in cui ogni intervento è accompagnato da una descrizione, le colonne coinvolte, il metodo pandas consigliato e una priorità (`high/medium/low`) basata sull’impatto sul Data Warehouse. Al termine del cleaning, per ogni tabella è stato inviato un secondo prompt: “*Narrativa audit per la tabella Olist {nome}*”, con le statistiche prima/dopo (null rimossi, duplicati eliminati, righe modificate). Il modello ha prodotto una breve narrativa in linguaggio naturale che descrive cosa è stato fatto, perché e quale miglioramento misurabile è stato ottenuto.

4 Progettazione Concettuale – Schema DFM

4.1 Scelta del fatto

Il fatto scelto è la **riga d’ordine** (`Order Item`), per tre ragioni:

- rappresenta l'evento atomico del dominio: l'acquisto di un singolo prodotto da parte di un cliente, presso un venditore, in una data precisa;
- permette analisi a grana fine per prodotto e venditore, impossibili aggregando al livello di ordine;
- tutte le misure di interesse (prezzo, spedizione, recensione, ritardo) sono associabili naturalmente a questo livello.

Granularità: una riga per ogni prodotto acquistato in ogni ordine. Lo schema ha **granularità transazionale**: ogni fatto corrisponde a un singolo evento di acquisto.

4.2 Attribute Tree

L'attribute tree è stato costruito espandendo ricorsivamente le dipendenze funzionali a partire dalle chiavi della tabella `Order_Items`, seguendo l'approccio data-driven. La Figura 11 mostra il risultato.

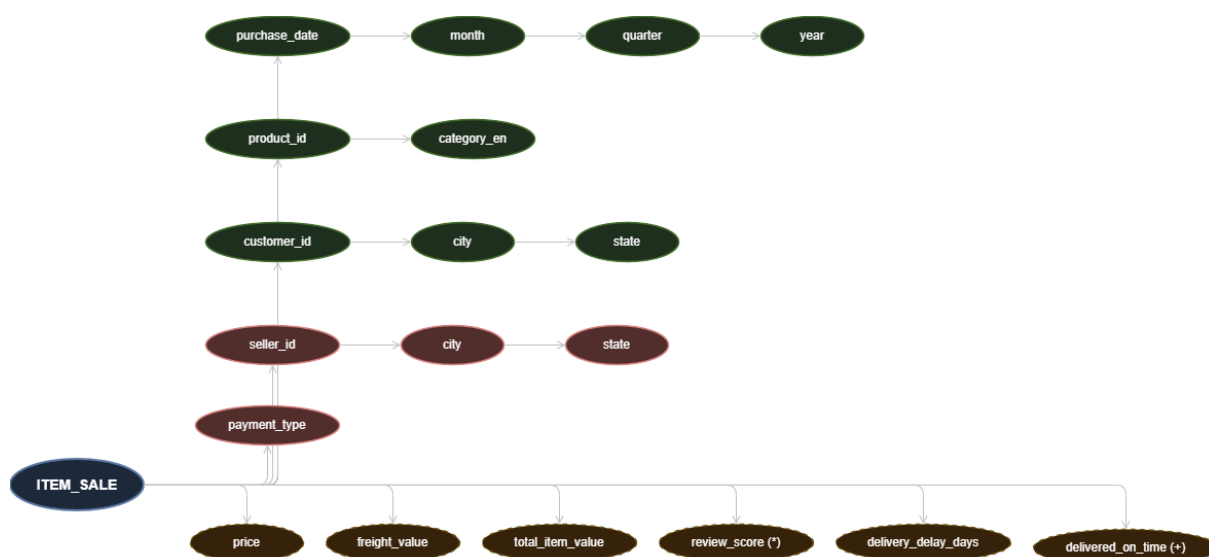


Figure 11: Attribute Tree della fact table `Order_Items`

4.3 Dall'Attribute Tree allo Schema DFM

Partendo dall'attribute tree, ho applicato le regole del metodo data-driven per ricavare lo schema DFM. I passaggi principali:

1. **Identificazione delle dimensioni:** le chiavi esterne (`product_id`, `seller_id`, `customer_id`, `payment_type`, `purchase_date`) sono state promosse a dimensioni.
2. **Editing dell'attribute tree:** secondo la metodologia DFM, l'editing dell'albero avviene tramite due operazioni. Il **pruning** di un vertice v consiste nella rimozione dell'intero sottoalbero radicato in v : gli attributi eliminati non saranno disponibili nella fact table né utilizzabili per aggregare i dati. Il **grafting** di un vertice v con

padre v' si usa invece quando v esprime un'informazione non interessante ma i suoi discendenti vanno mantenuti: i figli di v vengono collegati direttamente a v' ed v viene eliminato; si perde così il livello di aggregazione di v , ma si preservano i livelli sottostanti. Ho applicato queste operazioni come segue:

- *Pruning di `customer_unique_id`*: rimozione dell'intero sottoalbero (vertice foglia, senza discendenti). A livello di singolo order item, `customer_id` è già sufficiente: includerlo sarebbe ridondante.
- *Grafting di `zip_code_prefix`*: il vertice `zip_code_prefix` viene eliminato e i suoi figli `city` e `state` vengono collegati direttamente al padre (`customer_id/seller_id`). Si perde la granularità per singolo CAP, troppo fine per le analisi previste, ma si preserva la gerarchia geografica `city` $\rightarrow$ `state`.
- *Pruning degli attributi testo di `Reviews`*: rimozione dell'intero sottoalbero dei campi testuali liberi, non strutturati e senza valore per analisi OLAP quantitative. Mantenuto solo `review_score` come misura.
- *Pruning dei flag di qualità*: rimozione dell'intero sottoalbero dei flag `_imputed/_outlier/_missing` – utili nel reconciled database per auditing, ma privi di significato analitico nel DWH.
- *Aggiunta gerarchia `category_en` $\rightarrow$ `macro_category`*: la sorgente espone solo la categoria in portoghese. Ho aggiunto la traduzione in inglese (già disponibile nel reconciled database) e una macro-categoria per abilitare operazioni di roll-up di alto livello. Non è un'operazione di pruning/grafting in senso stretto, ma un'estensione dell'albero con un nuovo attributo derivato e una nuova gerarchia.
- *Promozione di `payment_type` a dimensione*: secondo le regole DFM le dimensioni si scelgono tra i figli diretti della radice dell'attribute tree. Un ordine può avere più pagamenti (es. voucher + carta): per evitare duplicazioni nella fact table, ho associato a ogni riga il **metodo di pagamento della prima rata registrata per l'ordine** nella tabella dei pagamenti riconciliata (`rec_fact_payments`), mentre `payment_value` rappresenta la somma di tutti gli importi pagati per quell'ordine. Si tratta di una scelta deterministica ma non basata sull'importo della singola rata: una possibile estensione futura è normalizzare i pagamenti multipli in una bridge table dedicata, evitando così la scelta di un singolo metodo “principale”.
- *Trattamento di `total_item_value` e `delivered_on_time`*: compaiono nell'attribute tree come misure concettuali. `total_item_value` = `price` + `freight_value` non è memorizzata nella fact table perché derivabile on-the-fly; viene calcolata direttamente negli strumenti di analisi (Tableau). `delivered_on_time` è derivabile da `delivery_delay_days` ≤ 0 : non richiede una colonna dedicata nel DWH.

3. Gerarchie di roll-up:

- **Tempo**: giorno $\rightarrow$ mese $\rightarrow$ trimestre $\rightarrow$ anno
- **Prodotto**: `product_id` $\rightarrow$ `category_en` $\rightarrow$ `macro_category`
- **Cliente**: `customer_id` $\rightarrow$ `city` $\rightarrow$ `state` $\rightarrow$ `region`
- **Venditore**: `seller_id` $\rightarrow$ `city` $\rightarrow$ `state`

- **Pagamento:** `payment_type` → `payment_category`

4. Classificazione delle misure in base all'additività:

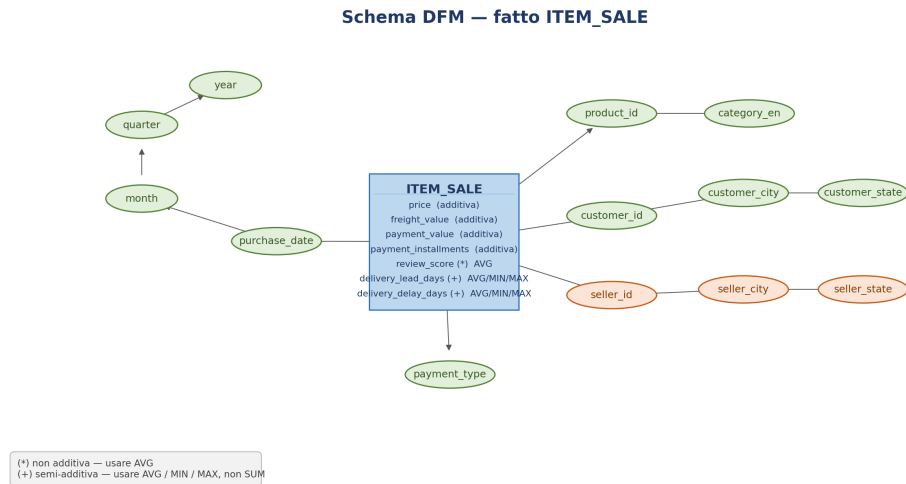
- *Additive:* `price`, `freight_value` – sommabili lungo tutte le dimensioni, incluso il livello item;
- *Additive solo a livello ordine:* `payment_value`, `payment_installments` – provengono da un aggregato per `order_id` (Sezione 6.2) e vengono replicate identiche su tutte le righe item dello stesso ordine in `fact_sales`; **non sono additive a livello item** (una SUM su più item dello stesso ordine duplica il valore). Vanno aggregate per `order_id` prima di applicare SUM, oppure usate con AVG a livello item;
- *Non additiva:* `review_score` – solo AVG, la somma dei punteggi non ha significato;
- *Semi-additive:* `delivery_delay_days`, `delivery_lead_days` – non sommabili nel tempo (la somma dei ritardi non ha significato), ma aggregabili con AVG, MAX, MIN.

4.4 Considerazioni sulla perdita di informazione

Due scelte progettuali comportano una perdita controllata di informazione:

- **Pagamenti multipli:** i pagamenti vengono aggregati per `order_id` (somma di `payment_value` e `payment_installments`, metodo di pagamento principale) prima del join con gli order-item. Si perde il dettaglio dei pagamenti secondari per metodo. Inoltre, poiché l'aggregato è per ordine ma viene riportato su ogni riga item, `payment_value` e `payment_installments` sono additivi solo a livello ordine, non a livello item (cfr. Sezione 6.2 e Tabella 6).
- **Geolocalizzazione:** aggregando per ZIP si perde la variabilità puntuale delle coordinate. Accettabile perché le analisi previste lavorano a livello città/stato. Da segnalare che 278 ZIP di clienti e 7 di venditori non hanno corrispondenza in `Geolocation`: per questi record, `lat` e `lng` risultano NULL nel DWH, senza impatto sulle analisi geografiche per stato.

Lo schema DFM risultante è in Figura 12.

Figure 12: Schema DFM del fatto `Order_Items`

5 Progettazione Logica — Star Schema

5.1 Traduzione dal DFM allo Star Schema

Ogni dimensione del DFM diventa una tabella con chiave primaria naturale. La fact table centralizza le FK verso le dimensioni e le misure. La scelta dello **star schema** rispetto allo snowflake è motivata dalla semplicità delle query OLAP: meno join, aggregazioni più veloci. Le gerarchie sono denormalizzate all'interno di ciascuna tabella di dimensione. Lo schema star è in Figura 13.

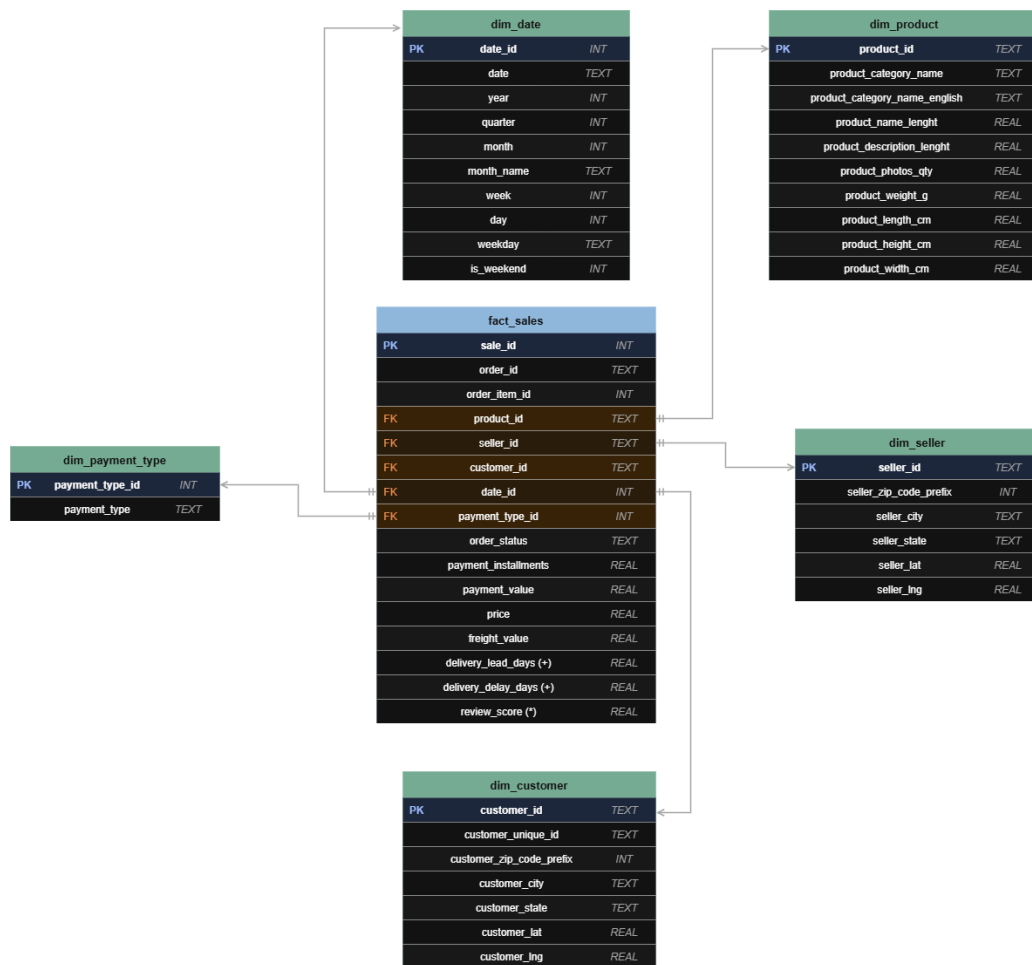


Figure 13: Star Schema del Data Warehouse Olist

5.2 Supporto alle operazioni OLAP

Il modello supporta le principali operazioni OLAP:

- **Roll-up:** da giorno a mese, da trimestre ad anno, da città a stato, da `product_category_name_english` a categoria;
- **Drill-down:** da anno a trimestre, da `category_en` a singolo prodotto;
- **Slice & Dice:** filtraggio per stato, anno, tipo di pagamento, categoria prodotto.

5.3 Tabelle del Data Warehouse

Table 5: Struttura della fact table `fact_sales`

Colonna	Tipo	Descrizione
<code>sale_id</code>	INT PK	Chiave surrogata della fact table
<code>order_id</code>	TEXT	Chiave naturale dell'ordine (tracciabilità)
<code>order_item_id</code>	INT	Numero progressivo dell'item nell'ordine
<code>date_id</code>	INT FK	Riferimento a <code>dim_date</code> (YYYYMMDD)
<code>product_id</code>	TEXT FK	Riferimento a <code>dim_product</code>
<code>customer_id</code>	TEXT FK	Riferimento a <code>dim_customer</code>
<code>seller_id</code>	TEXT FK	Riferimento a <code>dim_seller</code>
<code>payment_type_id</code>	INT FK	Riferimento a <code>dim_payment_type</code>
<code>order_status</code>	TEXT	Stato dell'ordine (attributo degenero)
<code>payment_installments</code>	REAL	Numero di rate del pagamento – additiva solo a livello ordine
<code>price</code>	REAL	Prezzo del prodotto – misura additiva
<code>freight_value</code>	REAL	Costo di spedizione – misura additiva
<code>payment_value</code>	REAL	Valore totale del pagamento – additiva solo a livello ordine (T)
<code>delivery_lead_days</code>	REAL	Giorni dalla vendita alla consegna – semi-additiva (AVG, MA)
<code>delivery_delay_days</code>	REAL	Ritardo rispetto alla data stimata (negativo = anticipo) – sem
<code>review_score</code>	REAL	Punteggio recensione 1–5 – solo AVG (non additiva)

Nota: `order_id` e `order_item_id` sono mantenuti nella fact table come attributi degeneri per garantire la tracciabilità verso la sorgente operativa, senza introdurre una dimensione dedicata. Le misure `payment_value` e `payment_installments` provengono da un aggregato per ordine e sono replicate su ogni riga item: sono quindi additive solo a livello ordine, non a livello item (cfr. Tabella 6 e Sezione 6.2).

Fact table: `fact_sales`

Tabelle di dimensione

`dim_date`(`date_id`, `date`, `year`, `quarter`, `month`, `month_name`, `week`, `day`, `weekday`, `is_weekend`)

Granularità giornaliera. `date_id` = YYYYMMDD (es. 20170913). Supporta roll-up day → month → quarter → year. 634 date distinte per il periodo 2016–2018.

`dim_product`(`product_id`, `product_category_name`, `product_category_name_english`, `product_name_lenght`, `product_description_lenght`, `product_photos_qty`, `product_weight_g`, `product_length_cm`, `product_height_cm`, `product_width_cm`)

Gerarchia: `product_id` → `product_category_name_english`. Nota: il livello `macro_category` è previsto nel DFM concettuale ma non implementato nella versione corrente dello star schema. I nomi di colonna `product_name_lenght` e `product_description_lenght` conservano il tipo originale della sorgente per compatibilità.

`dim_customer`(`customer_id`, `customer_unique_id`, `customer_zip_code_prefix`, `customer_city`, `customer_state`, `customer_lat`, `customer_lng`)

Gerarchia geografica: `city` → `state`. Nota: il livello `region` è previsto nel DFM concettuale ma non implementato nella versione corrente.

dim_seller(seller_id, seller_zip_code_prefix, seller_city, seller_state, seller_lat, seller_lng)
Gerarchia geografica: city → state.

dim_payment_type(payment_type_id, payment_type)
5 valori: credit_card (73,9%), boleto (19,0%), voucher (5,6%), debit_card (1,5%), not_defined (0,003%). Collegata alla fact table tramite il metodo di pagamento principale per ogni ordine (aggregato per order_id), evitando duplicazioni dovute a ordini con più metodi di pagamento.

5.4 Glossario delle misure

Table 6: Glossario delle misure della fact table `fact_sales`

Misura	Tipo / Aggregazione	Descrizione
<code>price</code>	Additiva – SUM, AVG, MIN, MAX	Prezzo unitario del prodotto. Sommabile lungo tutte le dimensioni (prodotto, cliente, venditore, tempo).
<code>freight_value</code>	Additiva – SUM, AVG	Costo di spedizione del singolo item. Sommabile lungo tutte le dimensioni.
<code>payment_value</code>	Additiva solo a livello ordine	Valore totale del pagamento per l'ordine. Attenzione: in <code>fact_sales</code> è replicato su ogni riga item dello stesso ordine (Sezione 6.2); una SUM a livello item sovrastima il totale (nel DWH: 19,23M contro 15,38M nei pagamenti originali). Aggregare per <code>order_id</code> prima di sommare, oppure usare AVG a livello item.
<code>payment_installments</code>	Additiva solo a livello ordine	Numero di rate del pagamento, replicato su ogni item dello stesso ordine per lo stesso motivo di <code>payment_value</code> . Aggregare per <code>order_id</code> prima di analisi sull'uso del credito per categoria/area geografica.
<code>delivery_lead_days</code>	Semi-additiva – AVG, MAX, MIN	Giorni dalla data di acquisto alla consegna effettiva. Non sommabile nel tempo; ha senso aggregare con AVG, MAX, MIN. NULL per ordini non consegnati.
<code>delivery_delay_days</code>	Semi-additiva – AVG, MAX, MIN	Ritardo rispetto alla data di consegna stimata (valore negativo = anticipo). Non sommabile nel tempo. NULL per ordini non consegnati.
<code>review_score</code>	Non additiva – AVG only	Punteggio recensione da 1 a 5. La somma è priva di significato: solo AVG è un'aggregazione semanticamente valida.
<code>COUNT(*)</code>	Conteggio – SUM	Numero di item venduti. Aggregabile lungo tutte le dimensioni; equivale al numero di transazioni a livello di granularità.

Nota sull'additività: le misure `delivery_delay_days` e `delivery_lead_days` sono incluse in `fact_sales` come misure semi-additive: aggregabili con AVG, MAX, MIN ma non con SUM nel tempo. Risultano NULL per gli ordini non ancora consegnati (null semantico,

non dato mancante).

6 Implementazione e Popolamento (ETL)

6.1 Architettura del processo ETL

Il passaggio dal reconciled database allo star schema segue il processo ETL (*Extract, Transform, Load*), eseguito per costruire il database SQLite finale `olist_dw.db`. Questa fase è implementata nello script `consegna/2_scripts/build_star_schema.py`, eseguibile a partire dai CSV puliti in `consegna/3_cleaned_data/`: lo script ricostruisce sia il reconciled layer (`rec_*`) sia il data warehouse layer (`dim_*` e `fact_sales`), garantendo la riproducibilità dell'intero processo descritto in questa sezione (in particolare le regole di aggregazione di `payment_type/payment_value` discusse in Sezione 6.2). L'architettura prevede due livelli distinti nel database SQLite:

- **Reconciled layer** (`rec_*`): contiene i dati puliti dal notebook di cleaning, caricati direttamente dai CSV. Costituisce il *operational data store* dal quale il processo ETL estrae i dati per costruire il data warehouse. Mantiene tutte le colonne di audit (flag `_imputed`, `_outlier`) e le misure derivate (`delivery_delay_days`, ecc.).
- **Data warehouse layer** (`dim_*` + `fact_sales`): star schema ottimizzato per query OLAP, privo dei flag di qualità e con gerarchie denormalizzate nelle dimensioni.

Il processo ETL si articola in tre fasi:

1. **Extract**: lettura dei CSV puliti e caricamento nel reconciled layer tramite `pandas.to_sql()` con `if_exists="replace"`.
2. **Transform**: costruzione delle tabelle di dimensione e della fact table a partire dal reconciled layer. In particolare:
 - Le tabelle `dim_*` vengono popolate per prime (no FK violation). Le chiavi naturali (`product_id`, `customer_id`, ecc.) sono mantenute come chiavi primarie, garantendo tracciabilità.
 - `dim_date` è generata sinteticamente estraendo le date distinte dai timestamp di acquisto (`rec_orders`), con calcolo di anno, mese, trimestre, giorno e giorno della settimana. Copre il periodo 2016–2018 (634 date distinte).
 - `dim_customer` e `dim_seller` sono arricchite con le coordinate geografiche (`lat/lng`) tramite LEFT JOIN con `rec_geolocation` su `zip_code_prefix`.
 - `fact_sales` viene popolata tramite JOIN tra `rec_order_items`, `rec_orders`, pagamenti aggregati per ordine (`payment_value` e `payment_installments` come somma su tutte le rate, `payment_type` come prima rata registrata) e media delle recensioni per ordine. Le chiavi naturali `order_id` e `order_item_id` sono mantenute come attributi degeneri.
3. **Load**: le tabelle trasformate vengono caricate nel DB finale (`olist_dw.db`) con vincoli di integrità referenziale abilitati (`PRAGMA foreign_keys = ON`). Il DB è pronto per essere interrogato con strumenti di analisi e visualizzazione (es. Tableau, SQLite-Viewer).

6.2 Dettaglio dei JOIN per la costruzione di fact_sales e delle dimensioni

La tabella `fact_sales` è costruita a partire da `rec_order_items` (una riga per prodotto per ordine) unendo quattro sorgenti del reconciled layer:

- **JOIN con `rec_orders` su `order_id`:** porta in `fact_sales` il `customer_id` e il `date_id` (ricavato convertendo `order_purchase_timestamp` nel formato intero YYYYMMDD).
- **LEFT JOIN con i pagamenti pre-aggregati:** prima del join, i pagamenti vengono aggregati per `order_id`: `payment_value` e `payment_installments` tramite SUM su tutte le rate dell'ordine (indipendentemente dal metodo), mentre `payment_type` viene preso dalla prima rata registrata per l'ordine (`first()`). Il LEFT JOIN garantisce che gli order-item senza pagamento registrato non vengano persi. **Effetto collaterale:** per gli ordini con più order-item, il totale aggregato per ordine viene replicato su ciascuna riga item di `fact_sales`. Di conseguenza `payment_value` e `payment_installments` risultano *additivi solo a livello ordine* e non a livello item (cfr. Tabella 6): la SUM di `payment_value` sull'intera `fact_sales` vale 19,23M contro i 15,38M dei pagamenti originali aggregati per ordine.
- **LEFT JOIN con le recensioni pre-aggregate:** le recensioni vengono aggregate per `order_id` con `AVG(review_score)`, in quanto un ordine può ricevere più valutazioni. Il LEFT JOIN preserva anche gli order-item privi di recensione.

Le tabelle di dimensione vengono costruite come segue:

- `dim_customer` e `dim_seller` vengono arricchite con le coordinate geografiche (`lat/lng`) tramite LEFT JOIN con `rec_geolocation` su `zip_code_prefix`. Il LEFT JOIN è essenziale: 278 ZIP di clienti e 7 di venditori risultano assenti in `rec_geolocation`; un INNER JOIN eliminerebbe quei record, producendo chiavi orfane in `fact_sales`.
- `dim_date` è generata sinteticamente: il codice estrae le date distinte da `rec_orders.order_purchase_timestamp` e calcola per ciascuna anno, mese, trimestre, giorno e giorno della settimana. La chiave primaria `date_id = YYYYMMDD` è un intero ordinabile e immediatamente leggibile (es. 20170913).
- `dim_payment_type` è ricavata dai valori distinti di `payment_type` presenti in `rec_fact_payments`, ai quali viene assegnata una chiave surrogata intera progressiva. Risultano 5 valori: *credit_card*, *boleto*, *voucher*, *debit_card*, *not_defined*.

6.3 Statistiche Finali del Data Warehouse

Table 7: Volumetrie finali dello Star Schema

Tabella	Record	Note
fact_sales	112.650	Tutte le righe d'ordine con FK operative valide
dim_customer	99.441	Un record per customer_id
dim_product	32.951	Include categoria in inglese
dim_seller	3.095	Include lat/lng da geolocation
dim_date	634	Periodo 2016–2018
dim_payment_type	5	credit_card, boleto, voucher, debit_card, not_defined

Integrità referenziale: verificata con query di controllo sul DB finale. La tabella fact_sales non presenta righe orfane verso dim_product, dim_seller, dim_customer e dim_date. Per dim_payment_type_id: 3 valori NULL legittimi, corrispondenti a ordini annullati prima della registrazione del pagamento; non costituiscono FK orfane ma riflettono uno stato operativo reale del dataset.